

Computer organisation project - Sieve of Eratosthenes

To test the performance of your CPU design, you are to use a program based on the “Sieve of Eratosthenes” – a method for finding prime numbers by removing all non-primes^[1]. The provided assembly code performs this algorithm on numbers in the range of 2 and 512. The result of this calculation is transmitted out of the memory mapped IO port.

The included VHDL component listens to the IO port and stores the transmitted prime numbers in a memory buffer. These numbers are then displayed on the Nexys 3 boards seven segment LED display.

Example program

An example implementation of the algorithm using Python and Numpy can be seen in figure 1. This outputs the prime number in both decimal and hexadecimal (figure 2).

```
#!/usr/bin/python

import numpy
n= 512
numbers=numpy.ones (n, dtype=bool)

for i in range(2,n):
    if numbers[i]:
        temp=i+i
        while temp<n:
            numbers[temp]=0
            temp=temp+i

print "prime number in decimal"
print numpy.where (numbers>0)

numpy.set_printoptions(formatter={'int':hex})
print "prime number in hexadecimal"
print numpy.where (numbers>0)
```

Figure 1 - Sieve of Eratosthenes in python

¹ [1] http://www.encyclopediaofmath.org/index.php/Eratosthenes,_sieve_of

```

prime number in decimal
(array([ 0, 1, 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31,
        37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89,
        97, 101, 103, 107, 109, 113, 127, 131, 137, 139, 149, 151, 157,
        163, 167, 173, 179, 181, 191, 193, 197, 199, 211, 223, 227, 229,
        233, 239, 241, 251, 257, 263, 269, 271, 277, 281, 283, 293, 307,
        311, 313, 317, 331, 337, 347, 349, 353, 359, 367, 373, 379, 383,
        389, 397, 401, 409, 419, 421, 431, 433, 439, 443, 449, 457, 461,
        463, 467, 479, 487, 491, 499, 503, 509])),)
prime number in hexadecimal
(array([0x0L, 0x1L, 0x2L, 0x3L, 0x5L, 0x7L, 0xbL, 0xdL, 0x11L, 0x13L,
        0x17L,
        0x1dL, 0x1fL, 0x25L, 0x29L, 0x2bL, 0x2fL, 0x35L, 0x3bL, 0x3dL,
        0x43L, 0x47L, 0x49L, 0x4fL, 0x53L, 0x59L, 0x61L, 0x65L, 0x67L,
        0x6bL, 0x6dL, 0x71L, 0x7fL, 0x83L, 0x89L, 0x8bL, 0x95L, 0x97L,
        0x9dL, 0xa3L, 0xa7L, 0xadL, 0xb3L, 0xb5L, 0xbfL, 0xc1L, 0xc5L,
        0xc7L, 0xd3L, 0xdfL, 0xe3L, 0xe5L, 0xe9L, 0xefL, 0xf1L, 0xfbL,
        0x101L, 0x107L, 0x10dL, 0x10fL, 0x115L, 0x119L, 0x11bL, 0x125L,
        0x133L, 0x137L, 0x139L, 0x13dL, 0x14bL, 0x151L, 0x15bL, 0x15dL,
        0x161L, 0x167L, 0x16fL, 0x175L, 0x17bL, 0x17fL, 0x185L, 0x18dL,
        0x191L, 0x199L, 0x1a3L, 0x1a5L, 0x1afL, 0x1b1L, 0x1b7L, 0x1bbL,
        0x1c1L, 0x1c9L, 0x1cdL, 0x1cfL, 0x1d3L, 0x1dfL, 0x1e7L, 0x1ebL,
        0x1f3L, 0x1f7L, 0x1fdL])),)

```

Figure 2 - output of python program

Implementation

The assembly and object code for the algorithm can be found in appendices 1 and 2. This program implements the generation of prime numbers and their transmission to the IO bus. You may make modifications to this program to solve issues such as data hazards.

You may briefly discuss the algorithm in your report, but your focus should be on the implementation of your CPU design to support the algorithm, rather than the algorithm itself.

VHDL module

The module has an interface for a CPU component that connects using four signals:

- Clk – a clock line
- loaddress – the address that the data should be sent to
- lodata – the data to be transmitted
- loenable – a write enable line

The data to be displayed is stored internally in a 32 bit x 128 entry array called “display_mem” the contents of this memory after a successful simulation run can be found in figure 3.

[illegible]

Figure 3 – display_mem state after succesful run

Appendix 1 – assembly code

```
--r0 - zero
--r1 - one
--r2 - working value
--r3 - target
--r4 - prime offset
--r5 - loop counter
--r6 - temp1
--r8 - sign test
load immediate r0 0
load immediate r1 1
load immediate r2 1
load immediate r3 512
load immediate r4 513
load immediate r8 2048

--Set the contents of working memory to 1
#label1
store r1 r2 0
inc r2 r2 r2
bne r2 r3 #label1

--Start the algorithm at index 1. Finish at 513

load immediate r2 1
#label2
inc r2 r2 r2
beq r2 r3 #label4
load r6 r2 0
bne r6 r1 #label2

--Send prime to the IO bus
store r2 r4 0
inc r4 r4 r4

--Clear all multiples of the prime
load immediate r6 0
add r6 r2 r6
#label3
add r6 r2 r6

--The calculated index may exceed the maximum value, but we lack a
branch on greater than. Subtract and check to see if the number is
negative
--this could be performed by checking the msb, but here we only test
bit 11 - which should be more significant than our mathematical
operations.
sub r6 r3 r7
and r7 r8 r7
beq r7 r8 #label2

--clear the non-prime value
store r0 r6 0
jump #label3
```

```
#label4  
jump #label4
```

Appendix 2 – object code

X"88000000", X"88440004", X"88880004", X"88cc01fc", X"89100200", X"8a202000", X"84840000", X"0088b800", X"108ffff8", X"88880004", X"0088b800", X"148c0034", X"8c980000", X"1187fff4", X"85080000", X"01113800", X"89980000", X"01898000", X"01898000", X"00d9c800", X"01e1d000", X"15e3ffd4", X"85800000", X"08120000", X"08180000",